Cascaded Autoregressive Diffusion Models for Remote Sensing Scene Generation

Yuanrui Zhang, Liqin Liu, *Member, IEEE*, Keyan Chen, *Graduate Student Member, IEEE*, Jia Xu, Zhenwei Shi, *Senior Member, IEEE*, and Zhengxia Zou, *Senior Member, IEEE*

***Abstract*—Recent advancements in generative models, such as generative adversarial networks (GANs) and diffusion-based models, have greatly improved the generation of realistic and diverse remote sensing images. In remote sensing images, the objects of interest (e.g., aircraft, ships, and vehicles) are often complexly coupled with the background of the scene, and there are also underlying distribution patterns among the objects. However, previous methods have lacked research on such issues, limiting their practical capability of generating scene images with complex coupling between the objects and the background. In this article, we propose a novel cascaded autoregressive framework that not only generates high-quality remote sensing images containing objects but also automatically predicts reasonable and detailed attributes, including objects' positions, sizes, and orientations. Unlike previous methods that generate all scene pixels simultaneously, we draw inspiration from large language models (LLMs) by treating the objects as "tokens" and the background as a "prompt," where the model learns to autoregressively predict the objects' attributes and then generate the object instances with a conditional reverse diffusion process. Experimental results demonstrate that our method can generate realistic, diverse, and controllable remote sensing scene images. At the same time, our method can generate corresponding box-level annotations for virtual objects to enhance the training of object detection models.**

***Index Terms*—Autoregressive diffusion models, cascaded pipeline, layout controllability, remote sensing scene generation.**

NOMENCLATURE

Notation	Description
z_0	Object-free background image.
z_t	z_0 with t objects generated.
χ_t	State of the tth object.
$\boldsymbol{r}_t$	Location of the tth object.
φ_t	Orientation of the tth object.
s_t	Size of the tth object.
$p(\chi_{1:T}\|z_0)$	Conditional probability of $\chi_{1:T}$ given z_0.
$p_\omega(\chi_{1:T}\|z_0)$	Modeled conditional probability.
$\boldsymbol{x}_{T_g}$	Starting point for object generation.
$\boldsymbol{x}_0$	Result for object generation.
$\boldsymbol{y}$	Orientation condition for object generation.
$\boldsymbol{x}_{\text{cond}}$	Background condition during object generation.
$\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_{t_g}, \boldsymbol{x}_{\text{cond}}, \boldsymbol{y}, t_g)$	Direct diffusion model output at step t_g.
α_{t_g}, $\bar{\alpha}_{t_g}$, and σ_{t_g}	Diffusion schedules and hyperparameters.

Received 21 April 2025; revised 5 August 2025 and 23 September 2025; accepted 10 October 2025. Date of publication 16 October 2025; date of current version 30 October 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62125102, Grant 62471014, Grant U24B20177, and Grant 623B2013; in part by Beijing Natural Science Foundation under Grant JL23005; and in part by the Fundamental Research Funds for the Central Universities. *(Corresponding author: Zhengxia Zou.)*

Yuanrui Zhang is with Shenyuan Honors College, Beihang University, Beijing 100191, China.

Liqin Liu, Keyan Chen, Zhenwei Shi, and Zhengxia Zou are with the Department of Aerospace Intelligent Science and Technology, School of Astronautics, and the State Key Laboratory of Virtual Reality Technology and Systems, Beihang University, Beijing 100191, China (e-mail: zhengxiazou@buaa.edu.cn).

Jia Xu is with Qian Xuesen Laboratory of Space Technology, China Academy of Space Technology, Beijing 100094, China.

Digital Object Identifier 10.1109/TGRS.2025.3622225

I. INTRODUCTION

IN RECENT years, generative models have demonstrated substantial advancements, achieving remarkable outcomes across various domains. A series of models based on generative adversarial networks (GANs) [1], [2], [3], [4], [5], diffusion [6], [7], [8], or autoregressive models [9], [10] can now generate realistic, diverse, imaginative, and high-quality images based on text or other metadata. These methods have also been extended to the generation of optical remote sensing images [11], [12], [13], [14], [15], [16], [17]. Existing models can generate high-resolution remote sensing images based on text [11], [13], [18], geographic information [11], resolution specifications [12], [19], and segmentation mask [14], [15], significantly expanding the boundaries of remote sensing image generation.

Although the aforementioned methods aim to enhance the generation quality of remote sensing scene images from multiple perspectives, they often struggle with a critical challenge—generating high-quality objects when they are complicatedly coupled with the background. This issue arises because objects occupy only a small fraction of the pixels, leading to background dominance during training. The current approach of generating the entire scene's pixels together makes it difficult to generate high-quality background and object images simultaneously. This often leads to blurry objects and their misplacement, especially in the generation of large-scale remote sensing images [11], [12].

At the same time, the objects of interest (e.g., aircraft, ships, and vehicles) are typically intricately coupled with the background, and there are underlying distribution patterns between the objects as well. Previous methods have seldom explored these issues. Specifically, the background in the remote sensing scene imposes constraints on the reasonable positions, sizes, and orientations of objects. For example, cars should be parked in parking spaces, and their sizes should roughly match those

1558-0644 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:14:08 UTC from IEEE Xplore. Restrictions apply.

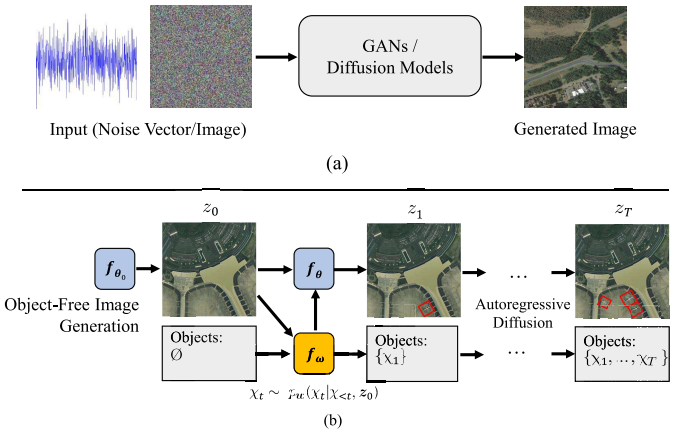


Fig. 1. Comparison between conventional remote sensing image generation approaches and our method: traditional remote sensing image generation methods employ GANs or diffusion models to transform noise from the latent space or pixel space into images, directly generating the entire scene. This approach often results in suboptimal object generation quality. In contrast, we adopt a cascaded generation pipeline, first generating a background image without objects, followed by the generation of the complete scene. During the full scene generation, we propose an autoregressive generation framework to progressively generate objects with correct layouts. We delineate the objects in the generated images with red bounding boxes. (a) Conventional remote sensing image generation. (b) Cascaded autoregressive diffusion model (ours).

of the spaces; aircraft should appear on the runway, moving in the direction of the runway, rather than on the roof beside it. In addition, there are potential distribution patterns between objects, such as the fact that two ships in close proximity at sea typically avoid converging trajectories and maintain a safe distance from each other. In this article, we propose a novel cascaded autoregressive framework that generates high-quality scene images containing objects. It is capable of reasonably generating and controlling fine-grained object attributes, including position, size, and orientation under multiobject coupling, while also supporting manual control and adjustment. We name our cascaded autoregressive diffusion models for remote sensing scene generation (CARD-RS). A simple comparison of our method with conventional remote sensing image generation methods is shown in Fig. 1.

To achieve high-quality object generation, our method first generates an object-free background image z_0 with a generative model f_{θ_0}, followed by the generation of objects to construct a complete scene. Notably, since we adopt a cascaded generation approach, the background image can either be generated or derived from real images. As a result, our method is naturally applicable to editing real images, transforming a series of real background images with few or no objects into object-controllable remote sensing scene images. This hybrid approach enables more realistic simulations, broadening the downstream applications of our method.

When generating objects, we first predict a reasonable layout, which includes the positions, sizes, and orientations of the objects—also referred to as the object states. Building on this, we perform the object generation through a reverse diffusion process.

Given the complex coupling relationships between the background and objects, as well as among objects themselves, inspired by large language models (LLMs) [20], [21], [22], we consider the background image as a prompt and the object's states as tokens, proposing an autoregressive generation framework. As illustrated in Fig. 1, we input z_0 along with the states of the previously generated $t-1$ objects, $\chi_{<t}$, into the layout generation model f_ω. This model predicts the distribution of the next object's state, $p_\omega(\chi_t|\chi_{<t}, z_0)$, from which a sample χ_t is drawn. The obtained state of the tth object, χ_t, is then fed into the diffusion model f_θ to generate the object, which we eventually embed into the original background, yielding the intermediate result z_t containing t objects. The entire process can be regarded as an autoregressive diffusion process.

We refer to the stage where objects are conditionally generated as diffusion-based object generation. To ensure that the objects and the background are well coordinated, we introduce background conditions when generating the objects via the diffusion model, helping the objects naturally blend into the background. Simultaneously, to better control the orientation of the objects, we employ the commonly used classifier-free guidance (CFG) [23] trick in diffusion models.

Furthermore, in addition to generating background-object images, our method can also generate box-level annotations, which can be used to boost the training of downstream image detection and recognition tasks. Virtual data produced by generative models have recently been utilized to enhance the accuracy of image recognition, object detection, and other perceptual tasks [24], [25], [26], [27], [28], [29]. However, although existing methods can synthesize high-fidelity images, they often lack access to object-level annotations, such as position, size, and orientation. This limitation hampers their effectiveness in large-scale generative data augmentation for object detection tasks. In contrast, our proposed method offers a distinct advantage: it not only generates high-quality scene images containing realistic objects but also simultaneously produces box-level annotations automatically, facilitating data augmentation for object detection models without additional manual labeling effort.

Experimental results demonstrate that our method can generate realistic, diverse, and controllable remote sensing scene images. It is also proven that the data we generate can truly help improve the accuracy of downstream tasks such as object detection. A comparison between our method and common scene generation techniques is provided in Table I. In this table, we mention label-image generation methods [30], [31], [32]. These approaches generate remote sensing scenes and produce corresponding labels, such as bounding boxes or segmentation masks. The core focus of these methods lies in exploring how generative models can enhance downstream tasks [34], [35], [36], [37]; thus, they represent only a small subset within the field of remote sensing image generation. Crucially, these methods generate labels by fitting existing label distributions, which inherently limits their capability to predict layouts from real-world background images. Consequently, they lack the capacity for editing real images, restricting their practical applications in image simulation scenarios.

Our contributions can be summarized as follows.

1) We propose a cascaded autoregressive diffusion model to address the problem of generating remote

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:14:08 UTC from IEEE Xplore. Restrictions apply.

TABLE I

COMPARISON OF EXISTING GENERATIVE MODELS. ✔ INDICATES THAT THE CORRESPONDING CAPACITY IS FULLY ATTAINABLE FOR SUCH METHODS, ✔✗ DENOTES EITHER INSUFFICIENT CAPACITY TO ACCOMPLISH THE TASK OR PARTIAL FAILURE AMONG SPECIFIC METHODS WITHIN THE CATEGORY, AND ✗ REPRESENTS THE COMPLETE ABSENCE OF SUCH CAPACITY ACROSS THE ENTIRE METHOD CATEGORY

Capability	Vanilla GANs or Diffusion [1], [6]	Conditional Generation [11]–[13], [15]	Label-Image Generation [30]–[33]	CARD-RS (ours)
High-fidelity image generation	✔	✔	✔	✔
Diverse image generation	✔	✔	✔	✔
Layout control	✗	✔✗	✔	✔
Object number and category controllability	✗	✔✗	✔✗	✔
Box-level annotation generation	✗	✗	✔✗	✔
Real image editing	✗	✔✗	✗	✔

sensing scenes with high-quality objects under complex object–background coupling.

2) We derive the loss function for autoregressive training during object layout generation with probabilistic modeling and propose a decoupled generation strategy for objects.
3) We achieve reasonable generation of object layouts, which can provide box-level annotations for the generated objects, and accurately control the refined attributes of the objects. Our experiments have shown that our generated images improve the performance in object detection tasks.

II. RELATED WORK

Generative models have recently achieved success in the field of remote sensing. In this section, we first introduce the widely applied fundamental methods of generative models, followed by an introduction to remote sensing scene generation methods.

A. Generative Models

Generative models are a class of machine learning models designed to learn the data distribution and generate new data samples that resemble the training data. The primary objective of a generative model is to learn a distribution p_θ, where $\boldsymbol{\theta}$ represents the parameters of the neural network, to approximate the true data distribution p_{data}.

Common generative models include GANs [1], autoregressive models [9], diffusion models [6], rectified flow Models [38], and so on. We provide a brief overview of them as follows.

1) Generative Adversarial Networks: GANs [1] consist of two neural networks: a generator and a discriminator, trained simultaneously through an adversarial process. After training, we use the generator to generate images. Various extensions have been developed based on the basic architecture, such as DCGAN [2], the StyleGAN series [4], [5], and CycleGAN [3]. Despite their success, GANs face several challenges. A well-known issue is mode collapse, where the generator produces limited output diversity. Another challenge is training instability, with model parameters often struggling to converge.

2) Autoregressive Model: Autoregressive models are a subset of generative models particularly well-suited for sequence prediction. These models use previous values in a sequence to predict future values by modeling the joint probability distribution of the sequence through the conditional probability formula

$$p(\chi_{1:t}) = p(\chi_1)\, p(\chi_2|\chi_1) \cdots p(\chi_t|\chi_{1:t-1}) \tag{1}$$

where χ_t denotes the value at timestep t. At each step, only $p(\chi_t|\chi_{1:t-1})$ needs to be predicted. In the field of generating images, there are two main paradigms: next-token prediction and next-scale prediction. Image GPT [9] predicts the next token in the pixel space, while VQ-GAN [39] predicts that in the discrete latent space. VAR [10] adopts the next-scale prediction paradigm. Recently, methods such as MAR [40] try to combine autoregressive models with diffusion models or rectified flow models to boost the application of autoregressive models in the continuous latent space.

3) Diffusion Models: Diffusion models are currently among the most prominent generative models, with the forward process viewed as adding noise in steps and the reverse process as predicting and removing noise. Ho et al. [6] proposed the denoising diffusion probabilistic model (DDPM), while Song et al. [41] introduced denoising diffusion implicit models (DDIMs), which accelerated the inference process of DDPM. Song et al. [42] improved the theory of diffusion models through stochastic differential equations (SDEs) and ordinary differential equations (ODEs), and in practice, Dhariwal and Nichol [43] first beat GANs with their guided diffusion models. Ho and Salimans [23] proposed classifier-free diffusion guidance, which inherited the idea of guided diffusion to better control synthetic data. To exploit image redundancy and accelerate generation, Rombach et al. [44] introduced latent diffusion models (LDMs), which helped diffusion models gain great popularity. Some large-scale models such as Imagen [8] and SDXL [7] have achieved great success in generating high-resolution images.

4) Rectified Flow Models: Rectified flow models [38] attempt to address the transformation problem between two distributions by learning the transformation velocity. Rectified flow models and diffusion models are closely related and, to some extent, can be transformed into each other. Rectified flow models have been proven to achieve relatively good performance in one-step generation.

B. Remote Sensing Image Generation

In the field of remote sensing image generation, numerous methods based on deep generative models have been

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:14:08 UTC from IEEE Xplore. Restrictions apply.

proposed for various image types, including optical images, hyperspectral images [45], [46], [47], [48], [49], and synthetic aperture radar (SAR) images [50]. The remote sensing images discussed in this article refer exclusively to optical remote sensing images.

To generate remote sensing images, quite a lot of approaches [14], [30], [51], [52] exploited GANs as the backbone. Wang et al. [51] used Graphical-GAN, Lv et al. [14] proposed D-sGAN, and Liu et al. [52] combined geometric transformation with GAN-based texture generation. Howe et al.'s work [30] employed GANs to first generate the entire segmentation masks, followed by generating remote sensing scene images consistent with these masks. Recently, diffusion models have also been applied to remote sensing scene generation. Toker et al. [31] introduced SatSynth to generate both images and corresponding masks for satellite segmentation [53] using diffusion models, and Tang et al. [32] introduced AeroGen to generate the bounding boxes and the remote sensing scene image based on these annotations with diffusion models. Yu et al. [12] introduced an unbounded remote sensing image generation method, and Sebaq and ElHelw [54] and Liu et al. [13] proposed text-conditioned diffusion models for remote sensing image generation, which hold the potential to boost the training of multiple interpretation tasks [55], [56], [57], [58]. Tang et al. [59] and Khanna et al. [11] presented diffusion-based remote sensing image generation models with multiple conditions.

There are also some methods focusing on synthesizing objects in remote sensing scenes [60], [61], [62], [63], [64], [65]; however, nearly all of these approaches are based on rendering simulations rather than generative models. Some methods [60], [61], [62], [63] involve rendering 3-D models of remote sensing objects into 2-D images to create novel remote sensing scenes. Xiao et al. [64] combined ship simulation samples with a neural style transfer network to enhance ship images. Liu et al. [65] synthesized aircraft images by rendering 3-D CAD models and subsequently improved their quality using CycleGAN [3]. Even when neural networks are used, they serve merely as auxiliary tools for rendering rather than as the core method. More critically, the layout of all objects relies on manual determination, making these methods highly costly and difficult to scale up.

Overall, remote sensing image generation methods based on generative models typically lack a focus on objects and seldom generate explicit layouts. As a result, they have difficulty creating high-quality objects, let alone ensuring reasonable object positions, sizes, and orientations within the generated scenes. Though some methods [30], [31], [32] can create the segmentation masks or bounding boxes, these labels are generated unconditionally rather than based on given backgrounds; thus, they cannot edit real remote sensing images. On the other hand, simulation-based methods that mainly generate remote sensing objects often rely on 3-D modeling and rendering, which limits their diversity, increases costs, and requires manual layout determination, thereby reducing efficiency. Our approach decouples object generation from background generation, specifically focusing on the generation of objects to ensure high-quality and controllable outcomes. We propose an autoregressive model to predict the object layout, automatically determining objects' positions, sizes, and orientations based on the background image. We will use this information as a foundation for generating objects. This process ultimately facilitates the generation of high-quality remote sensing scene images.

III. Methodology

Our cascaded approach consists of two primary stages. In the first stage, a generative model $\boldsymbol{f}_{\theta_0}$ generates a background image z_0. In the second stage, we perform autoregressive scene generation conditioned on z_0. Specifically, the scene generation process begins with an autoregressive layout generation model. Based on the generated layout, we extract relevant image patches from the background and apply diffusion models for conditional object generation. Finally, the generated objects are seamlessly integrated into the background, producing high-quality remote sensing scenes. The entire generation pipeline is illustrated in Fig. 2.

A. Overall Framework

In this section, we clarify the notations and introduce the overall framework of our method.

We denote the object-free background image as z_0 and see z_t as z_0 with t generated objects. We assume that z_0 should ultimately contain T objects after generation. We denote the state of the tth object in a remote sensing scene as $\chi_t = (\boldsymbol{r}_t, \varphi_t, s_t)$, where $\boldsymbol{r}_t$, φ_t, and s_t represent the position, orientation, and size of the object, respectively. Throughout this article, we consistently use the pixel coordinate system, with the coordinates generally denoted by $\boldsymbol{r}_t = [u_t, v_t]^{\mathrm{T}}$. The orientation of the object, also referred to as the angle, is measured counterclockwise from the horizontal axis pointing to the right. The position of an object refers to the coordinates of its center point. The size is defined as $s_t = 2\max(w_t, h_t)$, where w_t and h_t are the width and height of the horizontal bounding box (HBB) that encloses the object for convenience in the object generation step. In our pipeline, we define $\boldsymbol{r}_t$, φ_t, and s_t to obtain all the information required for object generation. The conditional probability distribution of $\chi_{1:T}$ given z_0 is $p(\chi_{1:T}|z_0)$, and layout generation means modeling it with $p_\omega(\chi_{1:T}|z_0)$, where $\boldsymbol{\omega}$ represents the parameters of the autoregressive layout generation model. We summarize the main notations used in this article in Nomenclature.

To ensure that the distribution modeled $p_\omega(\chi_{1:T}|z_0)$ closely approximates the true data distribution $p(\chi_{1:T}|z_0)$, we minimize the cross-entropy to form the objective function

$$\min_{\omega} E_{z_0\sim p(z_0)} E_{\chi_{1:T}\sim p(\chi_{1:T}|z_0)} - \log p_\omega\left(\chi_{1:T}|z_0\right). \tag{2}$$

To model the complex coupling relationships among objects, we employ an autoregressive model, utilizing the fact that

$$p_\omega\left(\chi_{1:T}|z_0\right) = \prod_{t=1}^{T} p_\omega\left(\chi_t|\chi_{<t}, z_0\right). \tag{3}$$

Note that, in this article, to make the expressions more concise, we stipulate that $p(\chi_1|\chi_{<1}, z_0) \equiv p(\chi_1|z_0)$ and

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:14:08 UTC from IEEE Xplore. Restrictions apply.

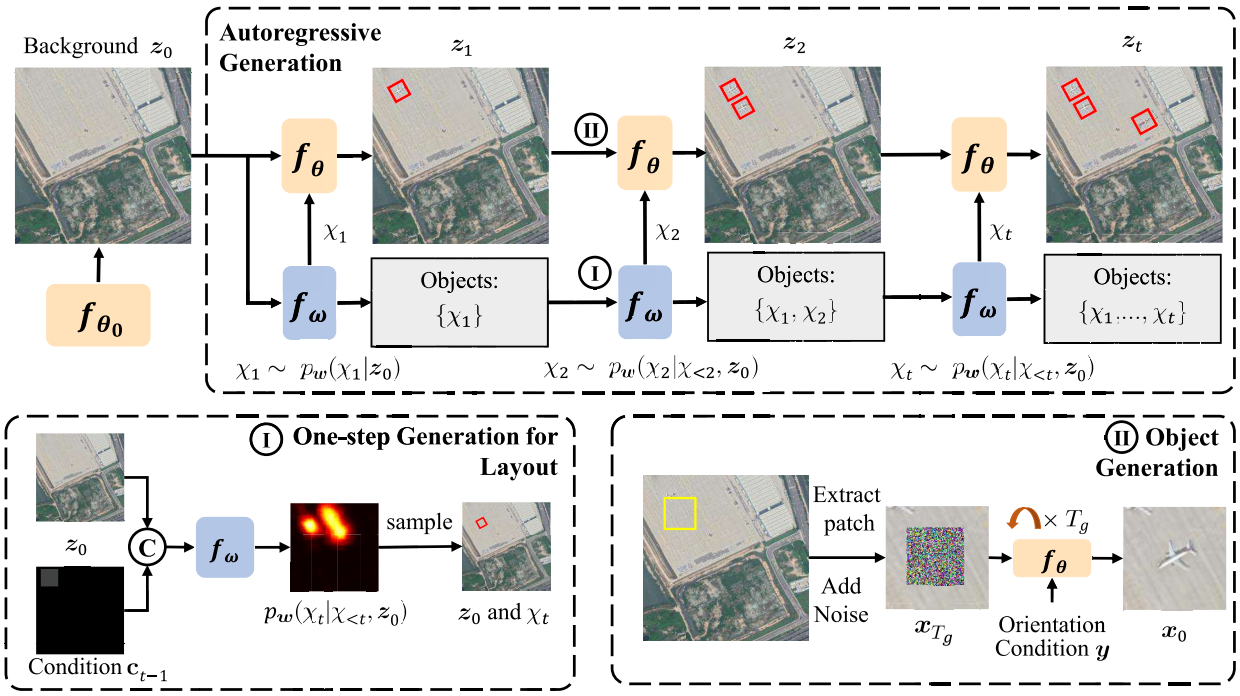


Fig. 2. Overview of our method: first, the background image z_0 is generated using the background generation model f_{θ_0}. Next, we employ an autoregressive approach to generate the complete scene image. During the autoregressive generation process, we begin by generating the layout. At each step, we concatenate z_0 and layout condition c_{t-1} and then feed them into the layout model f_ω, obtaining the predicted distribution $p_\omega(\chi_t|\chi_{<t}, z_0)$ from which we sample χ_t, as illustrated in part I. Note that in part I, © means the operation of concatenation in the channel dimension. For object generation, we extract the corresponding patch from the remote sensing background image where the object is expected to be located. The central part of the patch (i.e., the exact location of the object) is replaced with noise to obtain x_{T_g}, which is then denoised based on orientation conditions y with diffusion model f_θ to generate x_0. Finally, x_0 is reinserted into the original region to obtain the scene image generated at each step. We employ red bounding boxes to specify the positions, sizes, and orientations of the objects, whereas the yellow bounding box denotes the image patch utilized during the object generation process.

$p_\omega(\chi_1|\chi_{<1}, z_0) \equiv p_\omega(\chi_1|z_0)$. We denote the condition of $\chi_{1:t}$ as c_t. We only need to predict $p_\omega(\chi_t|\chi_{<t}, z_0)$ at each step, thereby reducing the complexity of the model's prediction task, and the objective becomes

$$\min_\omega \; E_{z_0\sim p(z_0)} \sum_{t=1}^{T} E_{\chi_{1:T}\sim p(\chi_{1:T}|z_0)} - \log p_\omega(\chi_t|\chi_{<t}, z_0). \tag{4}$$

Note that

$$\begin{aligned} &E_{\chi_{1:T}\sim p(\chi_{1:T}|z_0)} - \log p_\omega(\chi_t|\chi_{<t}, z_0) && (5)\\ &= E_{\chi_{1:t}\sim p(\chi_{1:t}|z_0)} E_{\chi_{t+1:T}\sim p(\chi_{t+1:T}|\chi_{1:t}, z_0)} - \log p_\omega(\chi_t|\chi_{<t}, z_0) && (6)\\ &= E_{\chi_{1:t}\sim p(\chi_{1:t}|z_0)} - \log p_\omega(\chi_t|\chi_{<t}, z_0) && (7)\\ &= E_{\chi_{<t}\sim p(\chi_{<t}|z_0)} E_{\chi_t\sim p(\chi_t|\chi_{<t}, z_0)} - \log p_\omega(\chi_t|\chi_{<t}, z_0). && (8) \end{aligned}$$

Therefore, we get the objective function

$$\min_\omega \; E_{z_0\sim p(z_0)} \sum_{t=1}^{T} E_{\chi_{<t}\sim p(\chi_{<t}|z_0)} E_{\chi_t\sim p(\chi_t|\chi_{<t}, z_0)} - \log p_\omega(\chi_t|\chi_{<t}, z_0). \tag{9}$$

Note that when $t = 1$, $\chi_{<t}$ becomes undefined and redundant, as no temporal dependencies exist prior to the first step. Nevertheless, we retain this unified notation throughout all steps $t = 1, \ldots, T$ to maintain mathematical consistency and simplicity in the overall formulation. We adopt this convention throughout the remainder of this article to maintain notational uniformity across all related expressions.

Since $\chi_t = (r_t, \varphi_t, s_t)$, the objective function can be further derived. φ_t and s_t follow a degenerate distribution concentrated at a single point given $r_t, \chi_{<t}, z_0$, which means that they are conditionally independent in the ground-truth distribution. Accordingly, we impose the same factorization in the model's predicted distributions such that

$$\begin{aligned} &p_\omega(\varphi_t, s_t|\chi_{<t}, z_0, r_t) \\ &= p_\omega(\varphi_t|\chi_{<t}, z_0, r_t)\, p_\omega(s_t|\chi_{<t}, z_0, r_t). \end{aligned} \tag{10}$$

Thus, the objective function can then be reformulated as

$$\begin{aligned} \min_\omega \; & E_{z_0\sim p(z_0)} \sum_{t=1}^{T} E_{\chi_{<t}\sim p(\chi_{<t}|z_0)} \\ &\{E_{r_t\sim p(r_t|\chi_{<t}, z_0)} - \log p_\omega(r_t|\chi_{<t}, z_0) \\ &\quad + E_{r_t\sim p(r_t|\chi_{<t}, z_0)} E_{\varphi_t\sim p(\varphi_t|\chi_{<t}, z_0, r_t)} - \log p_\omega(\varphi_t|\chi_{<t}, z_0, r_t) \\ &\quad + E_{r_t\sim p(r_t|\chi_{<t}, z_0)} E_{s_t\sim p(s_t|\chi_{<t}, z_0, r_t)} - \log p_\omega(s_t|\chi_{<t}, z_0, r_t)\}. \end{aligned} \tag{11}$$

As shown in (11), the objective function can be divided into three components: location loss, size loss, and orientation loss. This stems from our definition of the object's state χ_t.

Optimizing the objective enables the model to learn the layout distribution $p(\chi_{1:T}|z_0)$, and sampling from the learned distribution $p_\omega(\chi_{1:T}|z_0)$ yields the layout of z_0. When generating the tth object based on χ_t, we extract an image patch centered at the predicted location r_t with side length s_t from

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:14:08 UTC from IEEE Xplore. Restrictions apply.

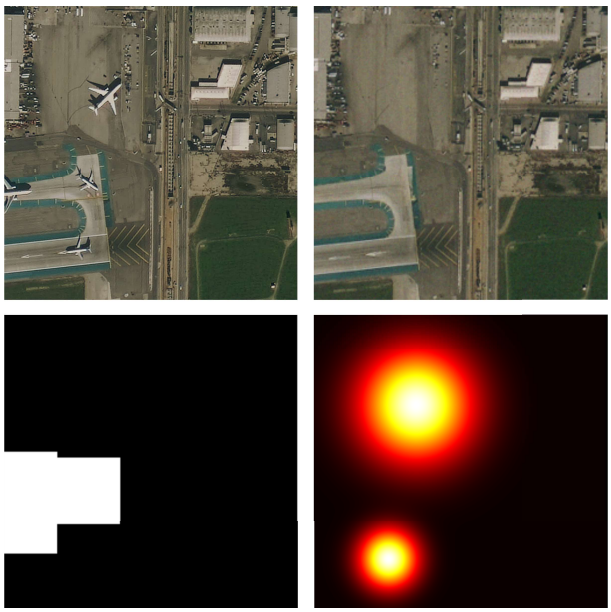


Fig. 3. (Top-left corner) Original image, (top-right corner) model input image z_0, (bottom-left corner) condition $\boldsymbol{c}_{t-1}$ when $t = 3$, and (bottom-right corner) corresponding ground truth of the location distribution in the form of a heatmap.

$\boldsymbol{z}_{t-1}$, mask its center with noise to obtain $\boldsymbol{x}_{T_g}$, and perform reverse diffusion

$$\boldsymbol{x}_{t_g-1} = \frac{1}{\sqrt{\alpha_{t_g}}}\left(\boldsymbol{x}_{t_g} - \frac{1-\alpha_{t_g}}{\sqrt{1-\bar{\alpha}_{t_g}}}\boldsymbol{\epsilon}_{\boldsymbol{\theta}}\left(\boldsymbol{x}_{t_g}, \boldsymbol{x}_{\text{cond}}, \boldsymbol{y}, t_g\right)\right) + \sigma_{t_g}\boldsymbol{\epsilon} \tag{12}$$

to denoise $\boldsymbol{x}_{t_g}$ at timestep t_g ($1 \leqslant t_g \leqslant T_g$), generating the object. Here, α_{t_g}, $\bar{\alpha}_{t_g}$, and σ_{t_g} are the diffusion model's schedules, $\boldsymbol{y} = \boldsymbol{y}(\varphi_t)$ represents the orientation condition, $\boldsymbol{x}_{\text{cond}}$ represents the background condition, $\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_{t_g}, \boldsymbol{x}_{\text{cond}}, \boldsymbol{y}, t_g)$ is the model output at timestep t_g, and $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. This patch is then placed back into its original position to form $\boldsymbol{z}_t$.

For each step $t = 1, 2, \ldots, T$, we iteratively perform the above operations, obtaining $\boldsymbol{z}_1, \boldsymbol{z}_2, \ldots, \boldsymbol{z}_T$, where the final output $\boldsymbol{z}_T$ corresponds to the generated remote sensing scene image we desire.

B. Autoregressive Layout Generation

We have introduced the objective function of our layout generation model. In this section, we will elaborate on some methodological refinements to enhance its applicability for model training. We will formalize the model's input–output framework and clarify the actual ground truth for our model.

To compute the objective function in (11), we first sample a background image $\boldsymbol{z}_0 \in \mathbb{R}^{3\times H\times W}$, where H and W denote the height and width of the input image, respectively. Then, we sample $\chi_{<t}$ from $p(\chi_{<t}|\boldsymbol{z}_0)$, which means that $t-1$ nonoverlapping objects are uniformly sampled from the ground-truth object state set $K = \{\kappa_1, \ldots, \kappa_T\}$, where κ_j represents the position, size, and orientation of the jth object. Next, we calculate the cross-entropy between the true conditional distribution and the estimated distribution of the tth object.

However, during training, we do not know the true distribution of the objects' states given a pure background image. Therefore, we use the LaMa model [66] to remove existing objects from remote sensing images, creating object-free background images as the model's input while also retaining the objects' state information K. We then utilize this information to sample $\chi_{<t}$ and calculate the ground truth for location distribution $p(\boldsymbol{r}_t|\chi_{<t}, \boldsymbol{z}_0)$, size distribution $p(s_t|\chi_{<t}, \boldsymbol{z}_0, \boldsymbol{r}_t)$, and angle distribution $p(\varphi_t|\chi_{<t}, \boldsymbol{z}_0, \boldsymbol{r}_t)$. The layout generation model requires not only the input background image $\boldsymbol{z}_0$ but also the condition for $\chi_{<t}$. In this article, a conditional map $\boldsymbol{c}_{t-1} \in \mathbb{R}^{1\times H\times W}$ is used to represent the conditional input. When there are no sampled objects, the conditional map is set to $\mathbf{0}$; otherwise, for each state $\chi_j = (\boldsymbol{r}_j, \varphi_j, s_j)$ in $\chi_{<t}$, the conditional map is set to 1 in a square region centered at $\boldsymbol{r}_j$ with a side length of s_j. Eventually, the input to the layout generation model is $\boldsymbol{z}'_{t-1} = \text{cat}[\boldsymbol{z}_0, \boldsymbol{c}_{t-1}]$, $\boldsymbol{z}'_{t-1} \in \mathbb{R}^{4\times H\times W}$. Here, cat[,] denotes concatenation in channel dimension. We visualize the conditional map in Fig. 3.

Theoretically, the position of the tth object should be uniformly selected from the remaining $T-t+1$ states $K \setminus \chi_{<t}$. However, this approach results in an extreme class imbalance, as, among thousands or even millions of potential positions, almost none corresponds to positive instances, making it challenging for the model to learn effectively. To address this issue, we modify the position distribution $p(\boldsymbol{r}_t|\boldsymbol{z}_0, \chi_{<t})$ when calculating the loss function as follows:

$$p\left(\boldsymbol{r}_t|\chi_{<t}, \boldsymbol{z}_0\right) = \frac{\sum_{i=1}^{T-t+1} p_i\left(\boldsymbol{r}_t\right)}{\sum_{\boldsymbol{r}_t}\sum_{i=1}^{T-t+1} p_i\left(\boldsymbol{r}_t\right)} \tag{13}$$

where $p_i(\boldsymbol{r}_t)$ represents the influence of the ith candidate object state from $K \setminus \chi_{<t}$ on the position distribution. Let the coordinates of the ith object state be $\boldsymbol{\mu}_i = [u_i^{(\text{gt})}, v_i^{(\text{gt})}]^{\text{T}}$, its size be $s_i^{(\text{gt})}$, and its angle be $\varphi_i^{(\text{gt})}$. We assume a Gaussian form for $p_i(\boldsymbol{r}_t)$

$$p_i\left(\boldsymbol{r}_t\right) = Z\exp\left\{-\frac{1}{2}\left(\frac{\left(u_t - u_i^{(\text{gt})}\right)^2}{\sigma_{i,1}^2} + \frac{\left(v_t - v_i^{(\text{gt})}\right)^2}{\sigma_{i,2}^2}\right)\right\}. \tag{14}$$

To ensure that the probability density decays rapidly as the distance from the center increases, we set $\sigma_{i,1} = \sigma_{i,2} = s_i^{(\text{gt})}/3$. To prevent smaller objects from having a larger probability density at the center, we set the normalizing factor Z to 1, meaning that $p_i(\boldsymbol{r}_t)$ is probably not a probability distribution. A visualization of the ground-truth set for $p(\boldsymbol{r}_t|\chi_{<t}, \boldsymbol{z}_0)$ is provided in Fig. 3.

Given $\boldsymbol{r}_t, \chi_{<t}, \boldsymbol{z}_0$, the orientation and size of the object are deterministic and are denoted as $\varphi(\boldsymbol{r}_t, \chi_{<t}, \boldsymbol{z}_0)$ and $s(\boldsymbol{r}_t, \chi_{<t}, \boldsymbol{z}_0)$, respectively. Utilizing $p_i(\boldsymbol{r}_t)$ from (14) as weights of $\varphi_i^{(\text{gt})}$ and $s_i^{(\text{gt})}$, we have

$$\varphi\left(\boldsymbol{r}_t, \chi_{<t}, \boldsymbol{z}_0\right) = \frac{\sum_{i=1}^{T-t+1} p_i\left(\boldsymbol{r}_t\right)\varphi_i^{(\text{gt})}}{\sum_{i=1}^{T-t+1} p_i\left(\boldsymbol{r}_t\right)} \tag{15}$$

$$s\left(\boldsymbol{r}_t, \chi_{<t}, \boldsymbol{z}_0\right) = \frac{\sum_{i=1}^{T-t+1} p_i\left(\boldsymbol{r}_t\right)s_i^{(\text{gt})}}{\sum_{i=1}^{T-t+1} p_i\left(\boldsymbol{r}_t\right)}. \tag{16}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:14:08 UTC from IEEE Xplore. Restrictions apply.

We simplify the problem by discretizing the ground truth of orientation φ_t and size s_t into 12 possible values each. For each position, the discrete values closest to the calculated $\varphi(\boldsymbol{r}_t, \chi_{<t}, z_0)$ and $s(\boldsymbol{r}_t, \chi_{<t}, z_0)$ are selected as the ground truth. The 12 possible values for φ_t are evenly spaced in the range $[0°, 330°]$, i.e., $\{0°, 30°, \ldots, 330°\}$. For s_t, the values are derived by performing K-means clustering [67] on the sizes from the training data. This discretization converts the regression problem into a classification problem, simplifying the task while preserving most of the relevant information about the objects and effectively capturing both their sizes and orientations. The orientations are not gained by clustering because the 12 evenly spaced values are sufficient to model the orientations of objects, and this uniform discretization aligns with the angle control required in the subsequent object generation part.

Once the input and the ground truth are defined, the model can be trained. For convenience, we denote the smallest unit in the objective function using symbols $L_{1,t}, L_{2,t}$ and $L_{3,t}$ defined as

$$L_{1,t} = E_{\chi_{<t}\sim p(\chi_{<t}|z_0)} E_{\boldsymbol{r}_t\sim p(\boldsymbol{r}_t|\chi_{<t},z_0)} - \log p_\omega\left(\boldsymbol{r}_t|\chi_{<t}, z_0\right) \tag{17}$$

$$L_{2,t} = E_{\chi_{<t}\sim p(\chi_{<t}|z_0)} E_{\boldsymbol{r}_t\sim p(\boldsymbol{r}_t|\chi_{<t},z_0)} E_{\varphi_t\sim p(\varphi_t|\chi_{<t},z_0,\boldsymbol{r}_t)} - \log p_\omega\left(\varphi_t|\chi_{<t}, z_0, \boldsymbol{r}_t\right) \tag{18}$$

$$L_{3,t} = E_{\chi_{<t}\sim p(\chi_{<t}|z_0)} E_{\boldsymbol{r}_t\sim p(\boldsymbol{r}_t|\chi_{<t},z_0)} E_{s_t\sim p(s_t|\chi_{<t},z_0,\boldsymbol{r}_t)} - \log p_\omega\left(s_t|\chi_{<t}, z_0, \boldsymbol{r}_t\right). \tag{19}$$

Then, the objective function L can be rewritten as $\sum_{t=1}^{T}\sum_{i=1}^{3}\lambda_i L_{i,t}$, where weights λ_i are introduced to balance the contributions of each loss term; we let $\lambda_1 = 10$ and $\lambda_2 = \lambda_3 = 1$ for practical considerations.

In practice, we find that the model with cross-entropy loss can be hard to train; we, therefore, use mse loss for training. We simply calculate the mse loss between $p_\omega(\boldsymbol{r}_t|\chi_{<t}, z_0)$ and $p(\boldsymbol{r}_t|\chi_{<t}, z_0)$ in $L_{1,t}$. MSE loss is a stable and efficient loss function [68], which shares the same optimal solution as the original loss. This implies that we can achieve stable convergence toward the optimal solution by utilizing mse loss. We apply the sigmoid function to the model's output logits to form $p_\omega(\boldsymbol{r}_t|\chi_{<t}, z_0)$; consequently, we also modify (13) to be

$$p\left(\boldsymbol{r}_t|\chi_{<t}, z_0\right) = \frac{\sum_{i=1}^{T-t+1} p_i\left(\boldsymbol{r}_t\right)}{\max_{\boldsymbol{r}_t}\sum_{i=1}^{T-t+1} p_i\left(\boldsymbol{r}_t\right)}. \tag{20}$$

With the modification to the objective, $p_\omega(\boldsymbol{r}_t|\chi_{<t}, z_0)$ and $p(\boldsymbol{r}_t|\chi_{<t}, z_0)$ are not probability distributions anymore, but, since they only differ from the true probability distribution by a normalization factor, for convenience, we still refer to them as position probability distributions.

C. Diffusion-Based Object Generation

Through the layout generation process, the object's expected position $\boldsymbol{r}$, orientation φ, and size s are determined. We then extract the corresponding patch from the background image and try to generate the object at the center of this patch. To achieve it, we consider this problem a diffusion-based inpainting task, where we replace the center of the image patch with Gaussian noise and denoise it conditioned on the corresponding background in the pixel space. We resize the extracted patch to $W_p \times H_p$, where W_p and H_p are fixed height and width.

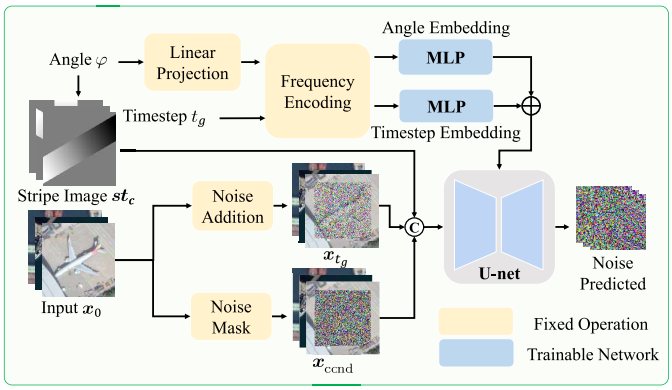


Fig. 4. Training for object generation: we replace the center of input $\boldsymbol{x}_0$ with a noise mask to create background condition $\boldsymbol{x}_{\text{cond}}$ and add noise to create $\boldsymbol{x}_{t_g}$. We create the stripe image condition $\boldsymbol{st_c}$ with angle φ and concatenate the $\boldsymbol{x}_{t_g}, \boldsymbol{x}_{\text{cond}}, \boldsymbol{st_c}$ together as the input to the model. We project the angle φ onto the same scale as the timestep t_g, apply frequency encoding to them, and pass each through an MLP. We then obtain the angle embedding and timestep embedding, which are summed and fed into the model. Finally, we predict the noise added to the center of $\boldsymbol{x}_0$. In this figure, © means the operation of concatenation in the channel dimension and ⊕ means the addition. The orange blocks represent fixed, parameter-free operations; blue blocks denote trainable network components, as we demonstrate in the figure.

Algorithm 1 Training for Object Generation

repeat
  $\boldsymbol{x}_0 \sim q(\boldsymbol{x}_0)$ ▷Sample $\boldsymbol{x}_0$ from the training data
  $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
  $\boldsymbol{x}_{\text{cond}} = \boldsymbol{mask} \odot \boldsymbol{\epsilon} + (\mathbf{1} - \boldsymbol{mask}) \odot \boldsymbol{x}_0$
  $t_g \sim \text{Uniform}(\{1, \ldots, T_g\})$ ▷Sample timestep t_g
  $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
  $\boldsymbol{x}_{t_g} = \sqrt{\bar{\alpha}_{t_g}}\boldsymbol{x}_0 + \sqrt{1 - \bar{\alpha}_{t_g}}\boldsymbol{\epsilon}$
  $\boldsymbol{x}_{t_g} = \boldsymbol{mask} \odot \boldsymbol{x}_{t_g} + (\mathbf{1} - \boldsymbol{mask}) \odot \boldsymbol{x}_0$
  ▷Add noise only to the center of $\boldsymbol{x}_0$
  $\boldsymbol{y} = \boldsymbol{y}(\varphi)$
  ▷Create condition based on the desired angle
  With probability p, set $\boldsymbol{y} \leftarrow \emptyset$
  ▷For the unconditional case
  Take gradient on
    $\nabla_{\boldsymbol{\theta}} \|\boldsymbol{mask} \odot (\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_{t_g}, \boldsymbol{x}_{\text{cond}}, \boldsymbol{y}, t_g))\|^2$
until converged

During training, we mask the central part of the clean image $\boldsymbol{x}_0$ with Gaussian noise to obtain the background condition $\boldsymbol{x}_{\text{cond}}$. To achieve it, we first generate a mask, denoted as $\boldsymbol{mask} \in \mathbb{R}^{3\times H_p \times W_p}$. At the center of the mask, the value is set to 1, while, at the edges, it is 0. Let the height of the central region where the value of 1 is h_c and the width is w_c. In this article, we set $h_c = 3/5H_p$ and $w_c = 3/5W_p$. The mask represents the part of the image that needs to be restored. Then, we sample from Gaussian noise $\boldsymbol{\epsilon} \in \mathbb{R}^{3\times H_p \times W_p}$ and define $\boldsymbol{x}_{\text{cond}}$ as

$$\boldsymbol{x}_{\text{cond}} = \boldsymbol{mask} \odot \boldsymbol{\epsilon} + (1 - \boldsymbol{mask}) \odot \boldsymbol{x}_0 \tag{21}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:14:08 UTC from IEEE Xplore. Restrictions apply.

where $\odot$ denotes the Hadamard product. We create the noisy image $\boldsymbol{x}_{t_g}$ mainly with

$$\boldsymbol{x}_{t_g} = \sqrt{\bar{\alpha}_{t_g}}\boldsymbol{x}_0 + \sqrt{1-\bar{\alpha}_{t_g}}\boldsymbol{\epsilon} \tag{22}$$

where $\bar{\alpha}_{t_g}$ is a hyperparameter and $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. We visualize noisy image $\boldsymbol{x}_{t_g}$ at timestep t_g and background condition $\boldsymbol{x}_{\text{cond}}$ in Fig. 4. We denote the condition that we use for controlling the angle φ as $\boldsymbol{y} = \boldsymbol{y}(\varphi)$. When applying CFG [23] to the training process, we randomly set the condition $\boldsymbol{y}$ to noncondition $\emptyset$ with probability p. We put $\boldsymbol{x}_{t_g}, \boldsymbol{x}_{\text{cond}}, \boldsymbol{y}, t_g$ into the network to predict the noise, and the loss is computed only over the central region. In this article, we use U-net [43] as the $\boldsymbol{\epsilon}$-predicting network. The training process is shown in Fig. 4. This method is mainly based on the open-source code of palette [69], except that we make the inpainting process controlled by condition $\boldsymbol{y}$ and apply CFG [23]. Details can be found in Algorithm 1.

The condition $\boldsymbol{y}$ is represented using a striped image $\boldsymbol{st_c} \in \mathbb{R}^{1\times H\times W}$ and an angle embedding $\boldsymbol{e_c}$. The striped image is applied in the pixel space. The pixel values along the length axis of the stripe follow a uniform distribution in the range $[-1, 1]$, with the head of the stripe having a value of 1 and the tail having a value of -1. The stripe's orientation corresponds to the desired object orientation φ. Pixels outside the stripe are set to 0, and for the unconditional case $\emptyset$, $\boldsymbol{st_c} = \mathbf{0}$. The visualization of the stripe is shown in Fig. 4. The main input to the network at each timestep is actually $\boldsymbol{x}'_{t_g} = \text{cat}[\boldsymbol{x}_{t_g}, \boldsymbol{x}_{\text{cond}}, \boldsymbol{st_c}] \in \mathbb{R}^{7\times H\times W}$ where cat[,] denotes concatenation in channel dimension. To obtain the angle embedding $\boldsymbol{e_c}$, we first linearly map the angle value (in degrees) to the range $[1, T_g]$ and then apply the same operation as timestep embedding. We add angle embedding to the timestep embedding $\boldsymbol{e}_{t_g}$ to better control the angle of the generated objects.

When sampling, the background image patch is first obtained. The central region of this patch is then replaced with standard Gaussian noise $\boldsymbol{\epsilon}$ to obtain the condition $\boldsymbol{x}_{\text{cond}}$, which also serves as the starting point $\boldsymbol{x}_{T_g}$. Next, the condition $\boldsymbol{y}$ is retrieved. Based on $\boldsymbol{x}_{t_g}, \boldsymbol{x}_{\text{cond}}, \boldsymbol{y}, t_g$, we apply CFG [23] to obtain the predicted noise

$$\boldsymbol{\epsilon}_{t_g} = (\eta+1)\,\boldsymbol{\epsilon}_{\boldsymbol{\theta}}\left(\boldsymbol{x}_{t_g}, \boldsymbol{x}_{\text{cond}}, \boldsymbol{y}, t_g\right) - \eta\boldsymbol{\epsilon}_{\boldsymbol{\theta}}\left(\boldsymbol{x}_{t_g}, \boldsymbol{x}_{\text{cond}}, \emptyset, t_g\right) \tag{23}$$

with the guidance intensity determined by η, where a larger η indicates stronger conditional control. We then denoise the image with

$$\boldsymbol{x}_{t_g-1} = \frac{1}{\sqrt{\alpha_{t_g}}}\left(\boldsymbol{x}_{t_g} - \frac{1-\alpha_{t_g}}{\sqrt{1-\bar{\alpha}_{t_g}}}\boldsymbol{\epsilon}_{t_g}\right) + \sigma_{t_g}\boldsymbol{\epsilon} \tag{24}$$

where α_{t_g}, $\bar{\alpha}_{t_g}$, and σ_{t_g} are the diffusion model's hyperparameters, and $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.

Combining layout generation and object generation, we demonstrate our autoregressive generation framework in Algorithm 2.

Algorithm 2 Autoregressive Generation

$z_0 \sim q(z_0)$ ▷Sample z_0 from its distribution
for $t = 1, \ldots, T$ **do**
  $\chi_t \sim p_\omega(\chi_t|\chi_{<t}, z_0)$
  Extract patch $\boldsymbol{x}_{T_g}$ with χ_t
  $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
  $\boldsymbol{x}_{T_g} = \boldsymbol{x}_{\text{cond}} = \boldsymbol{mask} \odot \boldsymbol{\epsilon} + (\mathbf{1} - \boldsymbol{mask}) \odot \boldsymbol{x}_{T_g}$
  $\boldsymbol{y} = \boldsymbol{y}(\varphi_t)$
  for $t_g = T_g, \ldots, 1$ **do** ▷DDIM can also be applied
    $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t_g > 1$, otherwise $\boldsymbol{\epsilon} = \mathbf{0}$
    Calculate $\boldsymbol{\epsilon}_{t_g}$ according to Eq. (23)
    $\boldsymbol{x}_{t_g-1} = \frac{1}{\sqrt{\alpha_{t_g}}}\left(\boldsymbol{x}_{t_g} - \frac{1-\alpha_{t_g}}{\sqrt{1-\bar{\alpha}_{t_g}}}\boldsymbol{\epsilon}_{t_g}\right) + \sigma_{t_g}\boldsymbol{\epsilon}$
    $\boldsymbol{x}_{t_g-1} = \boldsymbol{mask} \odot \boldsymbol{x}_{t_g-1} + (\mathbf{1} - \boldsymbol{mask}) \odot \boldsymbol{x}_{\text{cond}}$
  end for
  Put $\boldsymbol{x}_0$ back into z_{t-1} to obtain z_t
end for

D. Implementation Details

1) Data Processing for Layout Generation: We use the training split of the FAIR1M dataset [70] as our training dataset for layout and object generation. It is a remote sensing object detection dataset with oriented bounding boxes (OBBs). As mentioned in Section III-B, we need to remove the objects from the original training images while retaining their information. The positions and sizes of the objects can be directly obtained from the annotations in the FAIR1M dataset. However, the orientations of the objects cannot be directly obtained, as the directions in which the objects are heading are unknown. To address this problem, we fine-tune a classifier based on a pretrained VGG backbone [71] to determine the orientations of the objects. We also add small Gaussian noise to the input image to prevent the network from learning the trace of object removal.

2) Sampling Strategy for Layout Generation: When sampling from our trained layout generation model, we can simply choose the location with the greatest probability, but that will restrict the diversity of our models. On the other hand, simply sampling from $p_\omega(\boldsymbol{r}_t|\chi_{<t}, z_0)$ might lead to unreasonable results. We can tackle this problem by only sampling from the candidates with the top k probability.

3) Data Processing for Object Generation: We select objects of appropriate size and position from FAIR1M and rotate them to specific angles, namely, $\{0°, 30°, \ldots, 330°\}$, totaling 12 classes. Then, we crop a square patch of size l_b centered on the object and resize it to $W_p \times H_p$. We define $l_b = 2\max(w_b, h_b)$, where w_b and h_b are the width and height of the HBB enclosing the object, respectively.

IV. Experiment

A. Experimental Setup

We generate remote sensing scenes containing three types of objects: aircraft, vehicles, and ships. We demonstrate the effectiveness of our innovative method through the generation of these representative object categories. In principle, our method does not limit the number of object categories for generation. Therefore, in addition to aircraft, ships, and vehicles, the proposed method can be extended to other remote sensing object categories. The data used for layout generation and object generation in this article are derived

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:14:08 UTC from IEEE Xplore. Restrictions apply.

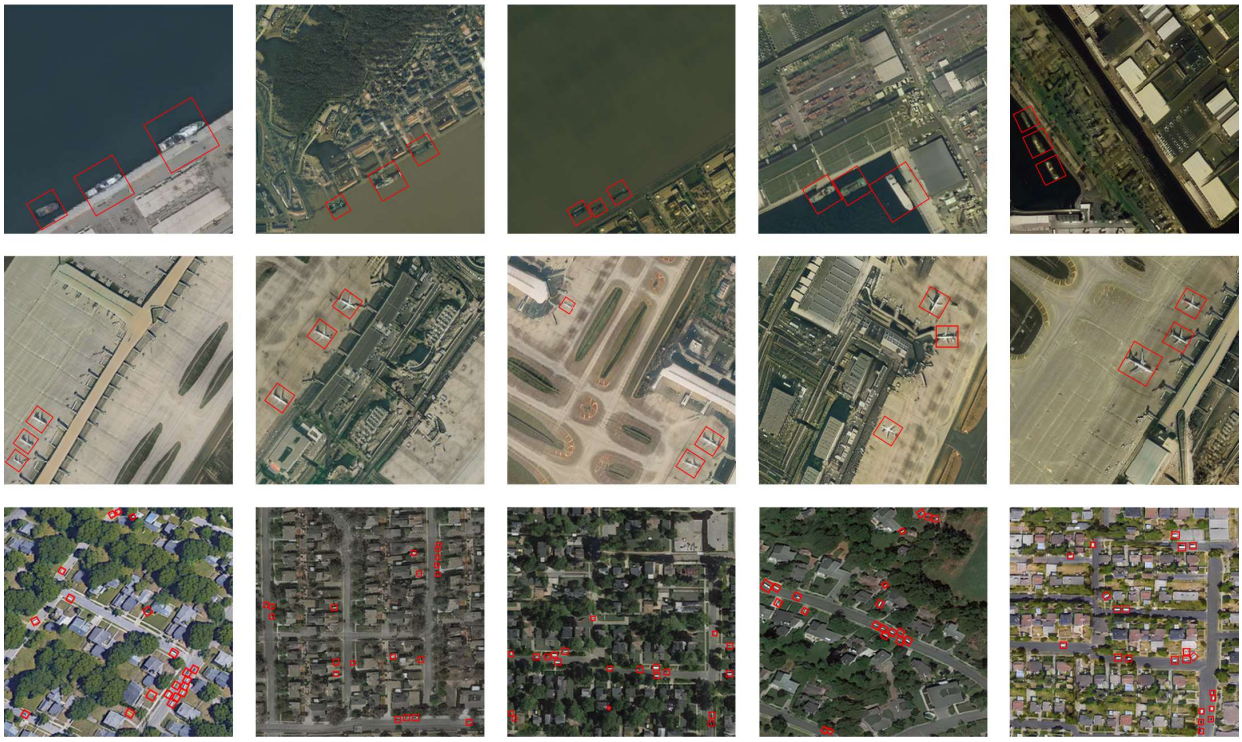


Fig. 5. Remote sensing scene generation results: we generate the virtual object-free backgrounds and then generate the scene. We specify the number of objects to be 3 for backgrounds to (top) generate ships and (middle) aircraft, and 20 for backgrounds to (bottom) generate vehicles. We have visualized the generated layouts using OBBs, indicating the objects' sizes, positions, and orientations.



Fig. 6. Remote sensing scene generation with real input: we generate virtual-real combined scenes for ships, aircraft, and vehicles. In this setup, besides employing different background sources, we also fix the number of vehicle objects at 30. The virtual-real combined generation results are arranged in the same manner as in Fig. 5 since all other settings remain unchanged apart from the background sources and the vehicle count.

from the FAIR1M dataset [70]. The spatial resolution of the dataset is relatively uniform, ranging from 0.3 to 0.8 m. Our layout model learns the scale relationship between objects and the background from the training data, thereby ensuring resolution consistency between the generated objects and the background.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:14:08 UTC from IEEE Xplore. Restrictions apply.



Fig. 7. Generation of complex composite scenes. Generated by randomly specifying the object combinations and their quantities, these scenes reveal that our method can generate complex scenes containing multiple types of objects. The label under each generated image indicates the selected combination and number of objects. Background images are from the test dataset.



Fig. 8. Comparison with other state-of-the-art models: given a predefined layout (where red bounding boxes indicate the expected object locations), we perform conditional generation. (From left to right) Generated results of AeroGen [32], MMO-IG [33], and CARD-RS (ours).

To demonstrate that our method can improve the accuracy of downstream object detection tasks, we apply our method for data augmentation on RetinaNet [72], Faster RCNN [73], Deformable DETR [74], and YOLOX [75]. Faster RCNN [73] serves as a classic two-stage object detection framework, employing a CNN-based architecture. RetinaNet [72] addresses the inherent class imbalance challenge in object detection through its focal loss mechanism, operating as a single-stage CNN-driven detector. Deformable DETR [74] integrates transformer architecture with convolutional operations, establishing an anchor-free paradigm that combines the hierarchical advantages of CNNs with global attention mechanisms. YOLOX [75] represents an important evolution in the YOLO series, transitioning to anchor-free detection while maintaining computational efficiency. This selected suite spans two-stage/single-stage detectors, anchor-based/anchor-free methodologies, and CNNs/transformer architectures, ensuring coverage of both established and cutting-edge approaches. Our evaluation employs the widely recognized average precision (AP) and AP50 as metrics. AP and AP50 are metrics used to evaluate the accuracy of object detectors. AP focuses on the model's localization accuracy by averaging precision values across multiple Intersection over Union (IoU) thresholds, from 0.50 to 0.95. AP50, in contrast, assesses performance at a single IoU threshold of 0.50, focusing on the model's ability to identify objects. A higher score is better for both metrics.

In the layout generation section, KL divergence and cross-entropy are used to measure the difference between the model's fit distribution and the real distribution. To evaluate the effectiveness of object generation, the metrics used are FID [76], sFID [77], precision, and recall [78], [79]. FID and sFID assess the difference between the distribution modeled by the generative model and the real distribution. Precision estimates how many of the generated images lie within the manifold of real images, while recall estimates how many of the real images fall within the manifold of generated images.

To quantitatively evaluate the control effect on the generated object's angle, we introduce the control rate (CR), defined as the ratio of generated samples with successfully controlled angles to all generated samples

$$\mathrm{CR} = \frac{\mathrm{TC}}{\mathrm{TC} + \mathrm{FC}} \tag{25}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:14:08 UTC from IEEE Xplore. Restrictions apply.

TABLE II
DATA AUGMENTATION RESULTS FOR OBJECT DETECTION

Category	RetinaNet [72]		Faster RCNN [73]		Deformable DETR [74]		YOLOX [75]	
	AP	AP50	AP	AP50	AP	AP50	AP	AP50
vehicle	4.3	15.5	0.7	1.9	18.2	49.0	19.8	55.8
vehicle + semi	**5.7**	**19.3**	**1.4**	**3.5**	21.5	55.9	20.3	56.2
vehicle + virtual	5.6	19.2	1.4	3.3	**21.6**	**56.1**	**20.7**	**57.0**
ship	12.3	28.5	16.4	35.1	18.1	42.3	17.4	40.1
ship + semi	**13.1**	**29.7**	**17.6**	**37.0**	**21.6**	**46.6**	**17.9**	**41.6**
ship + virtual	12.7	29.6	17.3	36.3	21.3	46.0	17.8	41.5
aircraft	52.2	77.3	54.3	77.9	53.6	78.8	57.1	81.0
aircraft + semi	**53.7**	**77.9**	**55.3**	**78.0**	**56.8**	**79.3**	57.8	**81.7**
aircraft + virtual	53.3	77.5	54.6	77.9	56.4	79.1	**57.9**	81.6
original (mean)	22.9	40.4	23.8	38.3	30.0	56.7	31.4	59.0
original + semi (mean)	**24.2**	**42.3**	**24.8**	**39.5**	**33.3**	**60.6**	32.0	59.8
original + virtual (mean)	23.9	42.1	24.4	39.2	33.1	60.4	**32.1**	**60.0**

where true control (TC) represents the number of successfully controlled samples and false control (FC) represents the number of unsuccessfully controlled samples. To calculate CR, we generate objects conditioned on randomly assigned orientations for 240 image patches each time. We then use manual evaluation to ensure the accuracy of the result.

B. Overall Generation Results

1) Qualitative Results: We train a StyleGAN3 model [5] on object-free training images to generate backgrounds and then perform autoregressive generation on the virtual backgrounds, generating remote sensing scene images that are realistic, diverse, and with controllable high-quality objects. Since our method employs a cascade generation approach, the background generation can be achieved by any mainstream generative models, including autoregressive models and diffusion models. The overall results of our method are shown in Fig. 5. Our method is capable of specifying the quantity and categories of objects, and the model will automatically determine reasonable positions, sizes, and orientations for these objects, generating them with high realism. In addition, our approach can also perform generation based on real background images, and the results are illustrated in Fig. 6. We generate these images with the test set, which is formed by removing the objects in the validation set of FAIR1M [70]. By generating on top of real backgrounds, our model can transform a large number of real background images into diverse remote sensing scene images with multiple objects, demonstrating significant potential for simulation applications. In addition to generating remote sensing scenes of single-category objects (see Fig. 6), our method can also create complex scenes with diverse object combinations, as depicted in Fig. 7. Furthermore, in Fig. 8, we compare our method with other state-of-the-art models. The results show that when generating from a given layout, our model achieves a visual quality comparable to that of state-of-the-art models [32], [33], despite its smaller model size.

TABLE III
DISTANCE BETWEEN THE GROUND TRUTH AND MODEL PREDICTION

Category	vehicle	aircraft	ship
KL divergence/nats	1.597	0.7993	1.832
Cross-entropy/nats	9.464	9.518	10.53

2) Verification on Downstream Object Detection Tasks: To verify the benefits of our method for the downstream task (i.e., object detection), we generate augmented images with a 1:1 quantity, matching the original FAIR1M training dataset [70]. To generate more precise labels, we estimate the length and width of the bounding box that encloses the object based on a combination of the object's size s_t, orientation φ_t, and the statistical average of its aspect ratio. Two augmentation configurations are implemented: hybrid scenes combining real backgrounds (object-free training split of FAIR1M) with virtual objects (denoted as "+semi") and fully generated virtual scenes (denoted as "+virtual"). We conduct object detection training under three configurations: original dataset, original + semi, and original + virtual, with evaluation on the FAIR1M validation set. Experiments are performed across four models: RetinaNet [72], Faster RCNN [73], Deformable DETR [74], and YOLOX [75]. All data are resized to 512×512 for model input.

According to our main results in Table II, we have the following findings.

1) Our augmented data improve the accuracy of object detection models. The maximum gains in mAP for our method on Retinanet, Faster RCNN, Deformable DETR, and YOLOX are 1.3, 1.0, 3.3, and 0.7, respectively, with the maximum mAP50 gains being 1.9, 1.2, 3.9, and 1.0. The performance improvement is most significant on Deformable DETR, with mAP increasing from 30.0 to 33.3 and mAP50 increasing from 56.7 to 60.6.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:14:08 UTC from IEEE Xplore. Restrictions apply.



Fig. 9. Model is not overfitting inpainting traces: we visualize the generated layouts and the ground truth. The black OBBs stand for the ground truth, where objects were before inpainting; the red OBBs indicate the generated layouts. If the model just overfits the inpainting traces, the black OBBs and red OBBs should overlap a lot, which is not the case. The model finds other reasonable regions.

2) RetinaNet, Faster R-CNN, and Deformable DETR generally perform better with the "+semi" configuration, while YOLOX does not exhibit a significant trend. We hypothesize this is related to the unique characteristics of each detector's standard training pipeline (e.g., the rule-based data augmentations in their standard implementations). The use of real backgrounds in the "+semi" setting, being more homogeneous with the test set, might offer a slight advantage for some models. Conversely, for other detectors such as YOLOX, the distinction between real and virtual backgrounds might be less significant for their training pipelines. Overall, while there is a slight inconsistency in the performance of "+semi" and "+virtual," the difference is not substantial. Most importantly, regardless of the specific detector used, both generated data types are proven to enhance performance, demonstrating the broad applicability and potential of our method for improving object detection models.
3) Retinanet and Faster RCNN perform poorly on vehicle object detection, mainly because we set the input size of the model to 512, and there are particularly small objects that are omitted in the training of Retinanet and Faster RCNN. However, Deformable DETR and YOLOX do not have this issue. Our experimental setup demonstrates that, regardless of whether small objects are neglected or not, our data augmentation method is effective.

C. Layout Generation and Object Generation

1) Qualitative and Quantitative Results for Layout: As shown in Figs. 5 and 6, we generate layouts for aircraft, vehicles, and ships. It is proven that our method can be adapted to different backgrounds with different object sizes and numbers. For the generation of vehicle layouts, the model achieves good performance in densely predicting 30 objects. To further verify that our models fit the training distribution, we calculate the KL divergence and cross-entropy between the generated location distribution and the ground truth using the training datasets. For each training image, we randomly choose

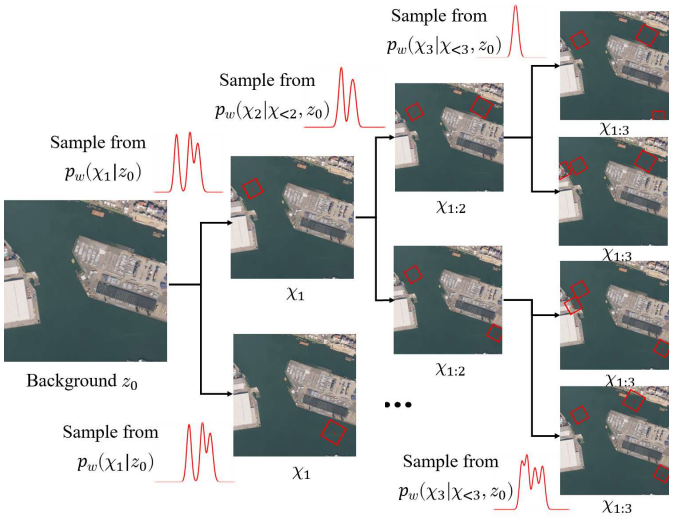


Fig. 10. Diversity analysis: we sample from the model twice at each step t, producing different results, bringing about the great diversity of the final products.

t objects to create the condition, obtain the ground truth, generate the layout, and calculate the metrics. We go through the datasets 30 times for aircraft and ship layout and 300 times for vehicle layout. The results are shown in Table III. The KL divergence is small, indicating that the distance between the two distributions is small.

2) Nonoverfitting Analysis for Layout: Since we use the inpainting images as our test sets, it would be natural to ask if the model is just learning and overfitting the inpainting traces. We compare the generated layouts and the layouts of the original images in Fig. 9, finding that they can be quite different, which indicates that our model is learning the right layouts of given images rather than the inpainting traces.

3) Layout Diversity Analysis: The diversity of our method originates from two sources: the noise added to the image before being given to the model and the top-k sampling method that we mention in Section III-D2. Fig. 10 shows that our strategy could produce reasonable and diverse results with fixed input images.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:14:08 UTC from IEEE Xplore. Restrictions apply.

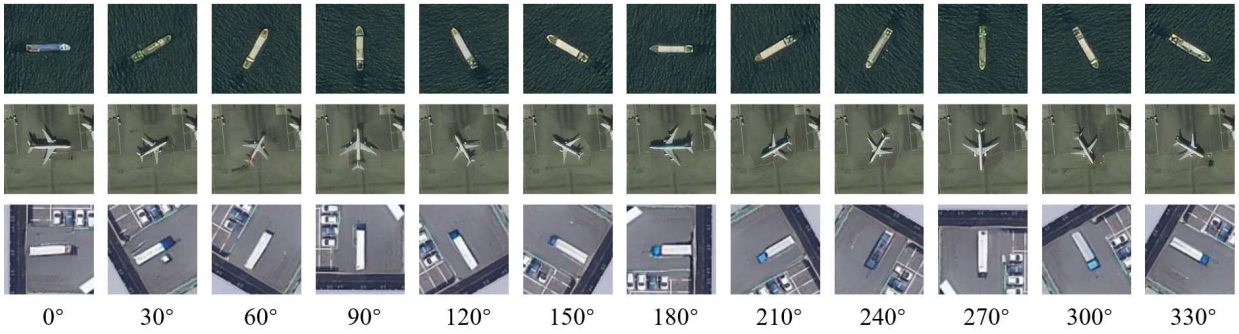


Fig. 11. Object generation: (from top to bottom) we generate ships, aircraft, and vehicles. The bottom row specifies the desired object angles, which are consistent across the subsequent figures.

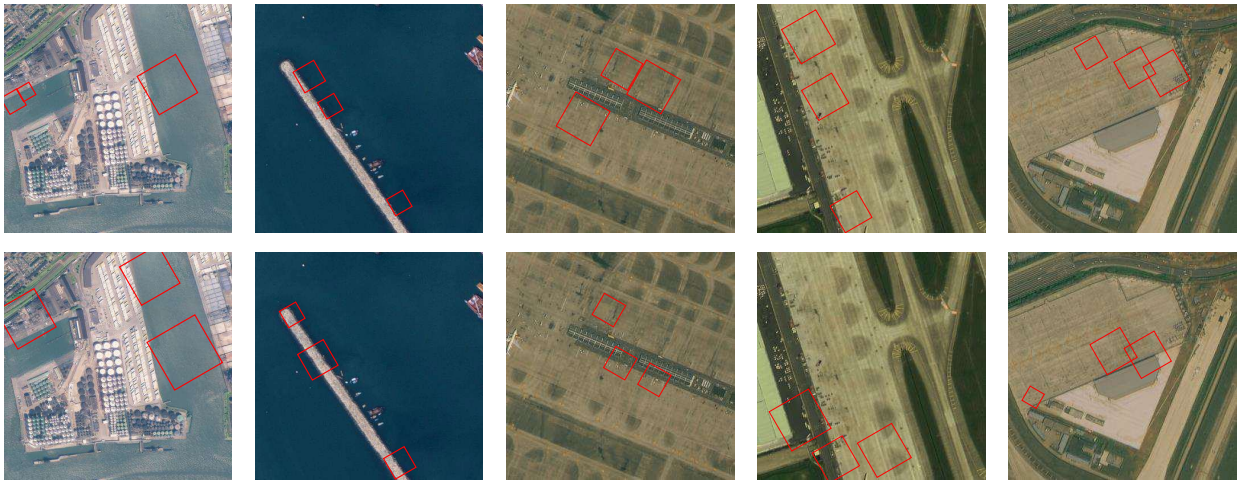


Fig. 12. Comparison between (top row) the Gaussian model and (bottom row) the uniform model.

TABLE IV
GENERATION QUALITY UNDER DIFFERENT GUIDANCE INTENSITIES

Category	η	FID	sFID	Precision	Recall
aircraft	4	19.31	24.01	0.6341	0.7883
aircraft	0	15.75	21.93	0.6960	0.8247
aircraft	-1	**13.48**	**20.48**	**0.7268**	**0.8611**
ship	4	**10.93**	**17.55**	0.6714	**0.8232**
ship	0	11.25	17.68	**0.6811**	0.8089
ship	-1	11.49	17.91	0.6618	0.8068
vehicle	4	11.54	**19.89**	0.6145	0.8635
vehicle	0	**11.32**	19.90	**0.6330**	**0.8760**
vehicle	-1	11.58	20.03	0.6135	0.8693

4) Object Generation Results: After sufficient training, our method can generate controllable and high-quality remote sensing object images. The results are shown in Fig. 11. In addition, we evaluate the FID, sFID, recall, and precision of the generative model under varying guidance intensities. The results are presented in Table IV. When $\eta = -1$, no orientation guidance is utilized during object generation; when $\eta = 0$, the CFG trick is not used; and when $\eta = 4$, a moderate level of guidance intensity is applied. From the table, we can observe that a larger η leads to a certain degree of degradation in the quality of aircraft generation, while it does not influence the quality of ship and vehicle that much.

D. Ablation Studies

1) Ground Truth for Layout Generation: When setting the ground truth for layout generation, to address the class imbalance problem in the position distribution, we choose a Gaussian distribution as the smoothing method, as shown in (14). The Gaussian distribution is selected for its favorable properties; its probability density function is smooth and naturally decays from the center. To demonstrate the reasonableness of our design, we compare this approach with a baseline model that uses an alternative smoothing technique: applying a uniform probability distribution within the bounding box of the ith candidate object. We train two models, designated the Gaussian model and the uniform model, which are identical in all aspects except for this ground truth smoothing method. For a fair comparison, we take the maximum probability for both models when predicting the distribution. Fig. 12 presents a representative qualitative comparison, demonstrating the superior performance of the Gaussian model over the uniform model. An analysis of the uniform model's failure cases suggests a significant cause: the uniform probability assignment across the entire bounding

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:14:08 UTC from IEEE Xplore. Restrictions apply.

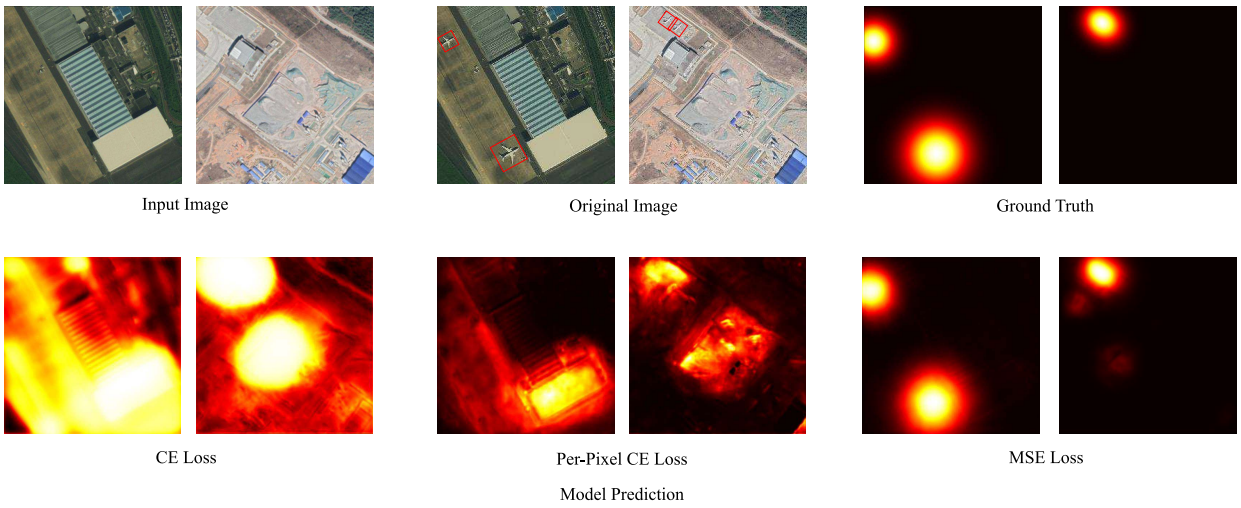


Fig. 13. Ablation for different loss functions: we visualize the location distribution fitting results for cross-entropy loss setting, per-pixel cross-entropy loss setting, and mse loss setting after 130k training steps. We present the input images, the original, un-inpainted remote sensing scene images, which can be considered the ground truth, the ground-truth location distributions, and the model's predicted results. For enhanced clarity, we mark the ground-truth object locations on the original image with red bounding boxes. The ground truth and the prediction results are shown in the form of heatmaps, with brighter areas indicating higher probabilities.

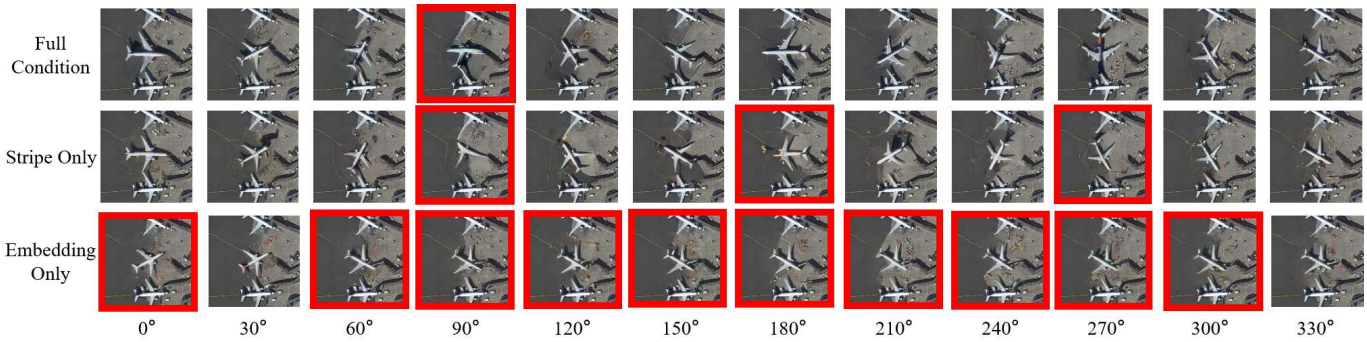


Fig. 14. Performance under different condition settings. (From top to bottom) The conditions are a stripe image with angle embedding, a stripe image only, and angle embedding only. Samples with control failures are highlighted in red boxes. It is evident that the combination of stripe image and angle embedding yields the best results. The stripe image alone achieves good performance but fails for angles of 90°, 180°, and 270°. In contrast, using only angle embedding leads to significantly inferior results.

TABLE V
CR UNDER DIFFERENT CONDITION SETTINGS

Category	Stripe Image with Angle Embedding	Stripe Image Only	Angle Embedding Only
aircraft	**97.9%**	92.5%	6.00%
ship	**99.6%**	99.2%	71.6%
vehicle	**95.8%**	94.6%	80.4%

box gives no preference to the center over the edges. This makes the model prone to placing an object's center near the bounding box boundary, leading to physically unreasonable outcomes, such as the center of a ship being positioned on the shore.

2) Loss for Layout Generation: To demonstrate the reasonableness of switching the objective function from cross-entropy loss to mse loss, we compare three formulations for $L_{1,t}$: mse loss, cross-entropy loss, and per-pixel cross-entropy loss. The ground truth $p(\boldsymbol{r}_t|\chi_{<t}, \boldsymbol{z}_0)$ for cross-entropy loss is the original definition in (13), while per-pixel cross-entropy loss shares the same definition of $p(\boldsymbol{r}_t|\chi_{<t}, \boldsymbol{z}_0)$ with mse loss as defined in (20). Per-pixel cross-entropy loss calculates the mean of the binary cross-entropy loss over each pixel of the input. We train models using each of these loss functions for 130k steps. Our results show that the per-pixel cross-entropy loss and cross-entropy loss settings are more difficult to train, with loss curves noisier and harder to converge. Also, they perform worse in fitting the training data compared to the mse loss setting, as shown in Fig. 13.

3) Condition for Object Generation: We also conduct ablation studies to verify the necessity of using the stripe image

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:14:08 UTC from IEEE Xplore. Restrictions apply.

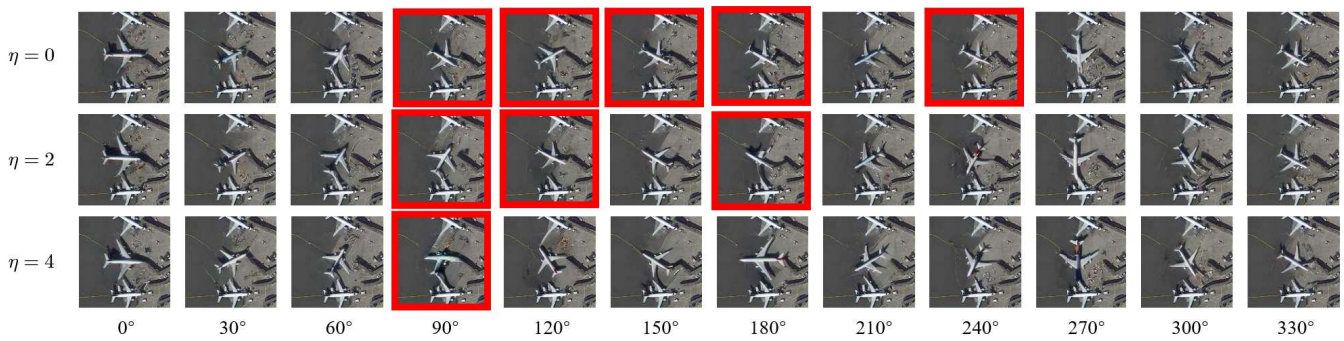


Fig. 15. CFG effect: background images with a significant influence on angle control are selected to demonstrate the effect under three guidance intensities. (From top to bottom) Values of η are 0, 2, and 4, respectively. We highlight the samples with control failures using red boxes. When η = 0, the model performs poorly at angles of 90°, 120°, 150°, 180°, and 240°. With η = 2, failures persist at 90°, 120°, and 180°. However, when η = 4, good control is achieved for all angles except 90°.

TABLE VI
CR UNDER DIFFERENT GUIDANCE INTENSITIES

η	4	2	0
CR	**97.9%**	95.8%	80.0%

combined with angle embedding as the condition for object generation. We train the model under three configurations: stripe image with angle embedding, stripe image only, and angle embedding only. We fix η = 4 for all models when sampling. The CR is measured with the models trained under these three configurations, with the results presented in Table V. As shown, the combination of stripe image and angle embedding achieves the best performance, while the stripe image alone also yields competitive results. In contrast, using only the angle embedding leads to significantly poorer performance. For ship and vehicle categories, the smaller performance drop under the angle embedding only setting can be attributed to common background cues—such as roads and embankments—which inherently guide the objects' orientations. These results underscore the effectiveness and necessity of combining the stripe image with the angle embedding for precise orientation control. Fig. 14 qualitatively illustrates this effect.

4) CFG Intensity: In this section, we demonstrate the important role of the CFG [23] trick in controlling the object's orientation. We validate it through aircraft generation. Background images containing objects are randomly cropped from the DOTA dataset [80], and random angles are assigned for object generation. The CR is evaluated for three different values of η (0, 2, and 4), as shown in Table VI. Notably, η = 0 corresponds to disabling CFG. The results confirm the effectiveness of CFG, which is further demonstrated qualitatively in Fig. 15.

V. CONCLUSION

In this article, we propose a cascaded autoregressive diffusion model for remote sensing scene generation. Our method decouples object generation from background generation, enabling enhanced focus on generating high-quality objects under complex object–background coupling. We employ an autoregressive model to predict object layout (i.e., position, size, and orientation) and precisely control these attributes through conditional reverse diffusion processes. Benefiting from cascaded architecture, our framework generates both fully virtual remote sensing scenes and hybrid scenes (real backgrounds + virtual objects), which enables not only high-fidelity simulation but also flexible real image editing. Experimental results demonstrate superior generation quality and prove that both scene types generated improve object detection performance, revealing significant potential for data augmentation applications besides simulation usages. We conduct quantitative and qualitative analyses to evaluate the effectiveness and diversity of our layout generation model. Furthermore, we present the performance of the object generation model along with quantitative evaluations. In the ablation study, we justify the rationality and necessity of our layout generation and object generation model designs. Our method offers a novel perspective on improving remote sensing image generation.

REFERENCES

[1] I. Goodfellow et al., "Generative adversarial networks," *Commun. ACM*, vol. 63, no. 11, pp. 139–144, 2020.
[2] A. Radford, L. Metz, and S. Chintala, "Unsupervised representation learning with deep convolutional generative adversarial networks," 2015, *arXiv:1511.06434*.
[3] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros, "Unpaired image-to-image translation using cycle-consistent adversarial networks," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 2242–2251.
[4] T. Karras, S. Laine, and T. Aila, "A style-based generator architecture for generative adversarial networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 4401–4410.
[5] T. Karras et al., "Alias-free generative adversarial networks," in *Proc. Annu. Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2021, pp. 852–863.
[6] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Proc. NIPS*, 2020, pp. 6840–6851.
[7] D. Podell et al., "SDXL: Improving latent diffusion models for high-resolution image synthesis," 2023, *arXiv:2307.01952*.
[8] C. Saharia et al., "Photorealistic text-to-image diffusion models with deep language understanding," in *Proc. NIPS*, vol. 35, 2022, pp. 36479–36494.
[9] M. Chen et al., "Generative pretraining from pixels," in *Proc. Int. Conf. Mach. Learn.*, vol. 1, 2020, pp. 1691–1703.
[10] K. Tian, Y. Jiang, Z. Yuan, B. Peng, and L. Wang, "Visual autoregressive modeling: Scalable image generation via next-scale prediction," 2024, *arXiv:2404.02905*.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:14:08 UTC from IEEE Xplore. Restrictions apply.

[11] S. Khanna et al., "DiffusionSat: A generative foundation model for satellite imagery," in *Proc. 12th Int. Conf. Learn. Represent.*, 2024, pp. 19539–19557.

[12] Z. Yu, C. Liu, L. Liu, Z. Shi, and Z. Zou, "MetaEarth: A generative foundation model for global-scale remote sensing image generation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 47, no. 3, pp. 1764–1781, Mar. 2025.

[13] C. Liu, K. Chen, R. Zhao, Z. Zou, and Z. Shi, "Text2Earth: Unlocking text-driven remote sensing image generation with a global-scale dataset and a foundation model," *IEEE Geosci. Remote Sens. Mag.*, vol. 13, no. 3, pp. 238–259, Sep. 2025.

[14] N. Lv et al., "Remote sensing data augmentation through adversarial training," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 14, pp. 9318–9333, 2021.

[15] Z. Yuan et al., "Efficient and controllable remote sensing fake sample generation based on diffusion model," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5615012.

[16] Z. Yu, C. Liu, C. Zhong, Z. Zou, and Z. Shi, "Multi-grained guided diffusion for quantity-controlled remote sensing object generation," *IEEE Geosci. Remote Sens. Lett.*, vol. 22, pp. 1–5, 2025.

[17] K. Chi, J. Li, J. Chu, Q. Li, and Q. Wang, "A diffusion model with physically plausible gradient for remote sensing shadow removal," *IEEE Trans. Geosci. Remote Sens.*, vol. 63, 2025, Art. no. 5639311.

[18] X. Gao et al., "RSVQ-diffusion model for text-to-remote-sensing image generation," *Appl. Sci.*, vol. 15, no. 3, p. 1121, Jan. 2025.

[19] B. Chen, K. Chen, M. Yang, Z. Zou, and Z. Shi, "Heterogeneous mixture of experts for remote sensing image super-resolution," *IEEE Geosci. Remote Sens. Lett.*, vol. 22, pp. 1–5, 2025.

[20] T. B. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 1877–1901.

[21] H. Touvron et al., "LLaMA: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.

[22] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," 2023, *arXiv:2307.09288*.

[23] J. Ho and T. Salimans, "Classifier-free diffusion guidance," 2022, *arXiv:2207.12598*.

[24] X. Li, J. Deng, and Y. Fang, "Few-shot object detection on remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5601614.

[25] X. Sun et al., "RingMo: A remote sensing foundation model with masked image modeling," *IEEE Trans. Geosci. Remote Sens.*, 2022, Art. no. 5612822.

[26] D. Wang et al., "MTP: Advancing remote sensing foundation model via multitask pretraining," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 17, pp. 11632–11654, 2024.

[27] K. Chen, C. Liu, B. Chen, W. Li, Z. Zou, and Z. Shi, "DynamicVis: An efficient and general visual foundation model for remote sensing image understanding," 2025, *arXiv:2503.16426*.

[28] K. Li et al., "SegEarth-R1: Geospatial pixel reasoning via large language model," 2025, *arXiv:2504.09644*.

[29] B. Chen, L. Liu, Z. Zou, and Z. Shi, "Target detection in hyperspectral remote sensing image: Current status and challenges," *Remote Sens.*, vol. 15, no. 13, p. 3223, Jun. 2023.

[30] J. Howe, K. Pula, and A. Reite, "Conditional generative adversarial networks for data augmentation and adaptation in remotely sensed imagery," *Proc. SPIE*, vol. 11139, pp. 119–131, Apr. 2019.

[31] A. Toker, M. Eisenberger, D. Cremers, and L. Leal-Taixé, "SatSynth: Augmenting image-mask pairs through diffusion models for aerial semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2024, pp. 27695–27705.

[32] D. Tang et al., "AeroGen: Enhancing remote sensing object detection with diffusion-driven data generation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2025, pp. 3614–3624.

[33] C. Yang, B. Zhao, Q. Zhou, and Q. Wang, "MMO-IG: Multiclass and multiscale object image generation for remote sensing," *IEEE Trans. Geosci. Remote Sens.*, vol. 63, 2025, Art. no. 5616412.

[34] Y. Li, Q. Hou, Z. Zheng, M.-M. Cheng, J. Yang, and X. Li, "Large selective kernel network for remote sensing object detection," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 16794–16805.

[35] Y. Liu, Q. Li, Y. Yuan, Q. Du, and Q. Wang, "ABNet: Adaptive balanced network for multiscale object detection in remote sensing imagery," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5614914.

[36] Y. Liu, W. Guo, C. Yao, and L. Zhang, "Dual-perspective alignment learning for multimodal remote sensing object detection," *IEEE Trans. Geosci. Remote Sens.*, vol. 63, 2025, Art. no. 5404015.

[37] Y. Zhang, M. Ye, G. Zhu, Y. Liu, P. Guo, and J. Yan, "FFCA-YOLO for small object detection in remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5611215.

[38] X. Liu, C. Gong, and Q. Liu, "Flow straight and fast: Learning to generate and transfer data with rectified flow," 2022, *arXiv:2209.03003*.

[39] P. Esser, R. Rombach, and B. Ommer, "Taming transformers for high-resolution image synthesis," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 12868–12878.

[40] T. Li, Y. Tian, L. He, M. Deng, and K. He, "Autoregressive image generation without vector quantization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 56424–56445.

[41] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," 2020, *arXiv:2010.02502*.

[42] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, "Score-based generative modeling through stochastic differential equations," 2020, *arXiv:2011.13456*.

[43] P. Dhariwal and A. Nichol, "Diffusion models beat GANs on image synthesis," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 8780–8794.

[44] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2022, pp. 10684–10695.

[45] L. Liu, W. Li, Z. Shi, and Z. Zou, "Physics-informed hyperspectral remote sensing image synthesis with deep conditional generative adversarial networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 3173532.

[46] B. Chen, L. Liu, C. Liu, Z. Zou, and Z. Shi, "Spectral-cascaded diffusion model for remote sensing image spectral super-resolution," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5528414.

[47] L. Liu, B. Chen, H. Chen, Z. Zou, and Z. Shi, "Diverse hyperspectral remote sensing image synthesis with diffusion models," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5532616.

[48] L. Pang et al., "HSIGene: A foundation model for hyperspectral image generation," 2024, *arXiv:2409.12470*.

[49] L. Pang, X. Rui, L. Cui, H. Wang, D. Meng, and X. Cao, "HIR-diff: Unsupervised hyperspectral image restoration via improved diffusion models," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2024, pp. 3005–3014.

[50] J. Guo, B. Lei, C. Ding, and Y. Zhang, "Synthetic aperture radar image synthesis by using generative adversarial nets," *IEEE Geosci. Remote Sens. Lett.*, vol. 14, no. 7, pp. 1111–1115, Jul. 2017.

[51] G. Wang, G. Dong, H. Li, L. Han, X. Tao, and P. Ren, "Remote sensing image synthesis via graphical generative adversarial networks," in *Proc. IGARSS - IEEE Int. Geosci. Remote Sens. Symp.*, Jul. 2019, pp. 10027–10030.

[52] Z. Liu, R. Guan, J. Hu, W. Chen, and X. Li, "Remote sensing scene data generation using element geometric transformation and GAN-based texture synthesis," *Appl. Sci.*, vol. 12, no. 8, p. 3972, Apr. 2022.

[53] K. Chen et al., "RSPrompter: Learning to prompt for remote sensing instance segmentation based on visual foundation model," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 4701117.

[54] A. Sebaq and M. ElHelw, "RSDiff: Remote sensing image generation from text using diffusion model," *Neural Comput. Appl.*, vol. 36, no. 36, pp. 23103–23111, Dec. 2024, Art. no. 5633520.

[55] C. Liu, R. Zhao, H. Chen, Z. Zou, and Z. Shi, "Remote sensing image change captioning with dual-branch transformers: A new method and a large scale dataset," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022.

[56] C. Liu, K. Chen, B. Chen, H. Zhang, Z. Zou, and Z. Shi, "RSCaMa: Remote sensing image change captioning with state space model," *IEEE Geosci. Remote Sens. Lett.*, vol. 21, pp. 1–5, 2024.

[57] C. Liu, R. Zhao, J. Chen, Z. Qi, Z. Zou, and Z. Shi, "A decoupling paradigm with prompt learning for remote sensing image change captioning," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5622018.

[58] K. Chen, B. Chen, C. Liu, W. Li, Z. Zou, and Z. Shi, "RSMamba: Remote sensing image classification with state space model," *IEEE Geosci. Remote Sens. Lett.*, vol. 21, pp. 1–5, 2024.

[59] D. Tang, X. Cao, X. Hou, Z. Jiang, J. Liu, and D. Meng, "CRS-diff: Controllable remote sensing image generation with diffusion model," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5638714.

[60] A. Kamilaris, C. van den Brink, and S. Karatsiolis, "Training deep learning models via synthetic data: Application in unmanned aerial vehicles," in *Proc. Int. Conf. Comput. Anal. Images Patterns*. Cham, Switzerland: Springer, 2019, pp. 81–90.

[61] Y. Yan, Z. Tan, and N. Su, "A data augmentation strategy based on simulated samples for ship detection in RGB remote sensing images," *ISPRS Int. J. Geo-Inf.*, vol. 8, no. 6, p. 276, Jun. 2019.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:14:08 UTC from IEEE Xplore. Restrictions apply.

[62] J. Shermeyer, T. Hossler, A. V. Etten, D. Hogan, R. Lewis, and D. Kim, "RarePlanes: Synthetic data takes flight," in *Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV)*, Jan. 2021, pp. 207–217.
[63] M. Kolos, A. Marin, A. Artemov, and E. Burnaev, "Procedural synthesis of remote sensing images for robust change detection with neural networks," in *Proc. 16th Int. Symp. Neural Netw.* Cham, Switzerland: Springer, 2019, pp. 371–387.
[64] Q. Xiao, B. Liu, Z. Li, W. Ni, Z. Yang, and L. Li, "Progressive data augmentation method for remote sensing ship image classification based on imaging simulation system and neural style transfer," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 14, pp. 9176–9186, 2021.
[65] W. Liu, B. Luo, and J. Liu, "Synthetic data augmentation using multiscale attention CycleGAN for aircraft detection in remote sensing images," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.
[66] R. Suvorov et al., "Resolution-robust large mask inpainting with Fourier convolutions," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis. (WACV)*, Jan. 2022, pp. 2149–2159.
[67] H.-P. Kriegel, E. Schubert, and A. Zimek, "The (black) art of runtime evaluation: Are we comparing algorithms or implementations?," *Knowl. Inf. Syst.*, vol. 52, no. 2, pp. 341–378, Aug. 2017.
[68] X. Mao, Q. Li, H. Xie, R. Y. K. Lau, Z. Wang, and S. P. Smolley, "Least squares generative adversarial networks," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 2794–2802.
[69] C. Saharia et al., "Palette: Image-to-image diffusion models," in *Proc. ACM SIGGRAPH Conf.*, 2022, pp. 1–10.
[70] X. Sun et al., "FAIR1M: A benchmark dataset for fine-grained object recognition in high-resolution remote sensing imagery," *ISPRS J. Photogramm. Remote Sens.*, vol. 184, pp. 116–130, Feb. 2022.
[71] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*.
[72] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 2980–2988.
[73] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 28, 2015, pp. 91–99.
[74] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, "Deformable DETR: Deformable transformers for end-to-end object detection," 2020, *arXiv:2010.04159*.
[75] Z. Ge, S. Liu, F. Wang, Z. Li, and J. Sun, "YOLOX: Exceeding YOLO series in 2021," 2021, *arXiv:2107.08430*.
[76] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "GANs trained by a two time-scale update rule converge to a local Nash equilibrium," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 6627–6638.
[77] C. Nash, J. Menick, S. Dieleman, and P. W. Battaglia, "Generating images with sparse representations," 2021, *arXiv:2103.03841*.
[78] M. S. M. Sajjadi, O. Bachem, M. Lucic, O. Bousquet, and S. Gelly, "Assessing generative models via precision and recall," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 5228–5237.
[79] T. Kynkäänniemi, T. Karras, S. Laine, J. Lehtinen, and T. Aila, "Improved precision and recall metric for assessing generative models," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 3904–3913.
[80] G.-S. Xia et al., "DOTA: A large-scale dataset for object detection in aerial images," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 3974–3983.



Yuanrui Zhang received the B.S. degree from Shenyuan Honors College, Beihang University, Beijing, China, in 2025. He is currently pursuing the Ph.D. degree with the School of Intelligence Science and Technology, Peking University, Beijing.

His research interests are machine learning and generative models.



Liqin Liu (Member, IEEE) received the B.S. and Ph.D. degrees in pattern recognition and intelligent systems from Beihang University, Beijing, China, in 2018 and 2024, respectively.

She is currently a Post-Doctoral Research Fellow with the School of Astronautics, Beihang University. Her research interests include hyperspectral image processing, remote sensing, and deep learning.



Keyan Chen (Graduate Student Member, IEEE) received the B.S. and M.S. degrees from the School of Astronautics, Beihang University, Beijing, China, in 2019 and 2022, respectively, where he is currently pursuing the Ph.D. degree with the Department of Aerospace Intelligent Science and Technology.

To date, he has authored over 40 peer-reviewed publications in leading journals and conferences, including PROCEEDINGS OF THE IEEE, IEEE TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE, and IEEE TRANSACTIONS ON GEOSCIENCE AND REMOTE SENSING. His research interests include remote sensing image processing, deep learning, pattern recognition, and multimodal. His website is https://chenkeyan.top



Jia Xu received the B.S. and Ph.D. degrees from Beihang University, Beijing, China, in 2001 and 2007, respectively.

He is currently the Chief Researcher of Qian Xuesen Laboratory of Space Technology, China Academy of Space Technology (CAST), Beijing. His research interests include new space system techniques and applications, and spacecraft system engineering.



Zhenwei Shi (Senior Member, IEEE) is currently a Professor and the Dean of the Department of Aerospace Intelligent Science and Technology, School of Astronautics, Beihang University, Beijing, China. He has authored or co-authored over 300 scientific articles in refereed journals and proceedings. His research interests include remote sensing image processing and analysis, computer vision, pattern recognition, and machine learning.

Prof. Shi serves as an Editor for IEEE TRANSACTIONS ON GEOSCIENCE AND REMOTE SENSING, *Pattern Recognition*, *ISPRS Journal of Photogrammetry and Remote Sensing*, and *Infrared Physics and Technology*. His personal website is https://levir.buaa.edu.cn/



Zhengxia Zou (Senior Member, IEEE) received the B.S. and Ph.D. degrees from Beihang University, Beijing, China, in 2013 and 2018, respectively.

From 2018 to 2021, he was a Post-Doctoral Research Fellow at the University of Michigan, Ann Arbor, MI, USA. He is currently a Professor at the Department of Aerospace Intelligent Science and Technology, School of Astronautics, Beihang University. He has published over 80 peer-reviewed papers in top-tier journals and conferences, including PROCEEDINGS OF THE IEEE, *Nature Communications*, IEEE TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE, IEEE/CVF COMPUTER VISION, and *Pattern Recognition*, with over 10 000 Google Scholar citations. His research interests include computer vision and related problems in remote sensing.

Dr. Zou serves as an Associate Editor for IEEE TRANSACTIONS ON IMAGE PROCESSING and IEEE TRANSACTIONS ON GEOSCIENCE AND REMOTE SENSING. His personal website is https://zhengxiazou.github.io/

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:14:08 UTC from IEEE Xplore. Restrictions apply.